

Experiment List 16-02-2026 To 21-02-2026

1) Given a integer matrix (or 2D array) $a[][]$ of dimensions $n * m$. Also, given another 2-D array $query[][]$ of dimensions $q * 4$.

For each index $0 < i < query.length$, find the sum of all the elements of the rectangular matrix whose top left corner is $(query[i][0], query[i][1])$ and bottom right corner is $(query[i][2], query[i][3])$.

Example -

Input:

$n = 3, m = 3, q = 2$

$a[][] = [$

$[1, 2, 3],$

$[4, 5, 6],$

$[7, 8, 9]$

$]$

$query[][] = [$

$[0, 0, 2, 2]$

$[1, 0, 2, 1]$

$]$

Ans)

The screenshot displays a coding platform interface. On the left, the 'Output Window' shows 'Compilation Results' for 'Custom Input' by 'Y.O.G.J. (AI Bot)'. It confirms the 'Problem Solved Successfully' with 214/214 test cases passed, 2/2 attempts correct, and 100% accuracy. The time taken is 0.27. A note states: 'You get marks only for the first correct submission if you solve the problem without viewing the full solution.' Below this, 'Solve Next' buttons are provided for 'Find kth element of spiral matrix', 'Make Matrix Beautiful', and 'Boolean Matrix'. The 'Stay Ahead With:' section is at the bottom.

On the right, the code editor shows a Java solution. It defines a `prefix` array and uses nested loops to calculate the sum for each query. The code is as follows:

```
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54

prefix[i][j] = a[i][j];
if (i > 0)
    prefix[i][j] += prefix[i - 1][j];
if (j > 0)
    prefix[i][j] += prefix[i][j - 1];
if (i > 0 && j > 0)
    prefix[i][j] -= prefix[i - 1][j - 1];
}

// Step 2: Answer queries
ArrayList<Long> result = new ArrayList<>();
for (int i = 0; i < q; i++) {
    int r1 = query[i][0];
    int c1 = query[i][1];
    int r2 = query[i][2];
    int c2 = query[i][3];

    long sum = prefix[r2][c2];
    if (r1 > 0)
        sum -= prefix[r1 - 1][c2];
    if (c1 > 0)
        sum -= prefix[r2][c1 - 1];
    if (r1 > 0 && c1 > 0)
        sum += prefix[r1 - 1][c1 - 1];
    result.add(sum);
}

return result;
```

2) Given a matrix a of size $n \times m$ which represents a park, there is some construction work needs to be done. You are also given q queries each query contains two numbers R and C , For every query we need to construct a footpath in the R th row and C th column, there is a cost of this construction, after the construction this path will divide the park into sections, and the cost of the construction is the sum of minimum value present in all the sections. You are asked to find this cost for all the queries.

Note: Elements present in queries array are according to 1-based indexing.

Example 1:

Input:

$n=3$

$m=3$

$a=\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\}$

$q=1$

$queries=\{\{2,2\}\}$

Ans)

The screenshot shows a coding platform interface with the following components:

- Header:** Search bar, navigation links (Courses, Tutorials, Practice, Jobs), and user profile icons.
- Left Sidebar:**
 - Problem Solved Successfully:** A green checkmark indicating the problem is solved.
 - Test Cases Passed:** 1122 / 1122.
 - Attempts:** 1 / 3.
 - Accuracy:** 33%.
 - Points Scored:** 4 / 4.
 - Your Total Score:** 63.
 - Solve Next:** Buttons for "Max sum in the configuration", "Sum of permutations", and "Rotate and delete".
 - Stay Ahead With:** A section titled "Build 21 Projects in 21 Days" with a subtext "Build real-world ML, Deep Learning & Gen AI projects".
- Main Area:**
 - Language:** Java (21).
 - Code Editor:** Contains a Java solution for the problem. The code iterates through queries, constructs a footpath, and calculates the cost based on the minimum value in the resulting sections.
 - Output Window:** Displays the result of the code execution.

3) Given an array $arr[]$, find the minimum number of operations required to make the sum of its elements less than or equal to half of the original sum. In one operation, you

may replace any element with half of its value (with floating-point precision).

Examples:

Input: `arr[] = [8, 6, 2]`

Ans)

The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with test cases passed (1115 / 1115), attempts (1 / 1), accuracy (100%), points scored (4 / 4), and time taken (0.3). Below this, there are buttons for 'Solve Next' (Majority Element, Fractional Knapsack, Minimum Platforms) and a section 'Stay Ahead With: Build 21 Projects in 21 Days'. On the right, the code editor shows a Java solution for a problem involving a max heap and a while loop to reduce the sum to half.

```
1 import java.util.*;
2
3 class Solution {
4     public int minOperations(int[] arr) {
5         // Max heap (store largest elements first)
6         PriorityQueue<Double> pq = new PriorityQueue<>(Collections.reverseOrder());
7
8         double sum = 0;
9
10        // Build heap and calculate sum
11        for (int num : arr) {
12            sum += num;
13            pq.add((double) num);
14        }
15
16        double target = sum / 2.0;
17        int operations = 0;
18
19        // Perform operations until sum is reduced to half
20        while (sum > target) {
21            double largest = pq.poll();
22            double half = largest / 2.0;
23            sum -= largest; // reduce sum by half of largest
24            sum += half;
25            pq.add(half);
26            operations++;
27        }
28        return operations;
29    }
30 }
31
32
33 }
```

4) You are given an array `arr[]` of size `n`, where `arr[i]` denotes the range of working hours a person at position `i` can cover.

- If `arr[i] ≠ -1`, the person at index `i` can work and cover the time interval `[i - arr[i], i + arr[i]]`.
- If `arr[i] = -1`, the person is unavailable and cannot cover any time.

The task is to find the minimum number of people required to cover the entire working day from 0 to `n - 1`. If it is not possible to fully cover the day, return -1.

Examples:

Input: `arr[] = [1, 2, 1, 0]`

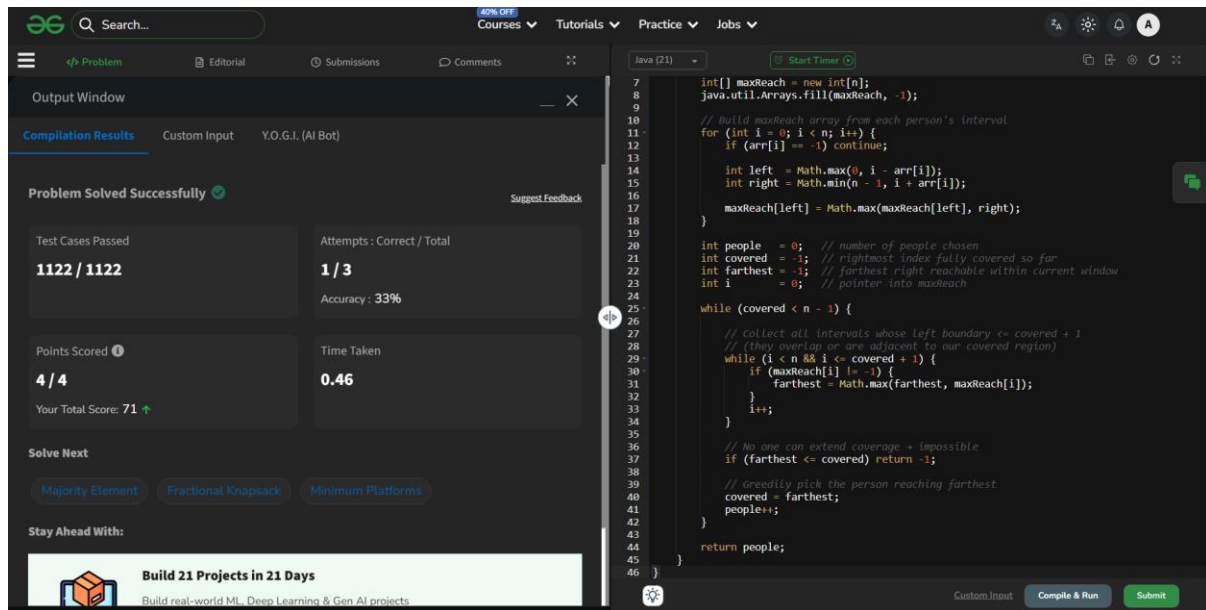
Output: 1

Explanation: The person at index 1 can cover the interval `[-1, 3]`. After adjusting to valid bounds, this becomes `[0, 3]`, which fully covers the entire working day 0 to `n - 1`.

Therefore, only 1 person is required to cover the whole day.

Input: `arr[] = [2, 3, 4, -1, 2, 0, 0, -1, 0]`

Ans)



The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. Below this, it shows 'Test Cases Passed: 1122 / 1122', 'Attempts: Correct / Total: 1 / 3', 'Accuracy: 33%', 'Points Scored: 4 / 4', and 'Your Total Score: 71'. At the bottom, it says 'Solve Next' with buttons for 'Majority Element', 'Fractional Knapsack', and 'Minimum Platforms'. On the right, the code editor shows Java code for a problem. The code defines an array 'maxReach' and iterates through it, calculating the previous smaller element for each element. The code is as follows:

```
7 int[] maxReach = new int[n];
8 java.util.Arrays.fill(maxReach, -1);
9
10 // Build maxReach array from each person's interval
11 for (int i = 0; i < n; i++) {
12     if (arr[i] == -1) continue;
13
14     int left = Math.max(0, i - arr[i]);
15     int right = Math.min(n - 1, i + arr[i]);
16
17     maxReach[left] = Math.max(maxReach[left], right);
18 }
19
20 int people = 0; // number of people chosen
21 int covered = -1; // rightmost index fully covered so far
22 int farthest = -1; // farthest right reachable within current window
23 int i = 0; // pointer into maxReach
24
25 while (covered < n - 1) {
26     // Collect all intervals whose left boundary <= covered + 1
27     // (they overlap or are adjacent to our covered region)
28     while (i < n && i <= covered + 1) {
29         if (maxReach[i] != -1) {
30             farthest = Math.max(farthest, maxReach[i]);
31         }
32         i++;
33     }
34
35     // No one can extend coverage = impossible
36     if (farthest <= covered) return -1;
37
38     // Greedily pick the person reaching farthest
39     covered = farthest;
40     people++;
41 }
42
43 return people;
44
45 }
46 }
```

5) ou are given an integer array `arr[ ]`. For every element in the array, your task is to determine its Previous Smaller Element (PSE).

The Previous Smaller Element (PSE) of an element `x` is the first element that appears to the left of `x` in the array and is strictly smaller than `x`.

Note: If no such element exists, assign `-1` as the PSE for that position.

Examples:

Input: `arr[] = [1, 6, 2]`

Output: `[-1, 1, 1]`

Explanation:

For 1, there is no element on the left, so answer is `-1`.

For 6, previous smaller element is 1.

For 2, previous smaller element is 1.

Input: `arr[] = [1, 5, 0, 3, 4, 5]`

Output: `[-1, 1, -1, 0, 3, 4]`

Explanation:

For 1, no element on the left, so answer is `-1`.

For 5, previous smaller element is 1.

For 0, no element on the left smaller than 0, so answer is -1.

For 3, previous smaller element is 0.

For 4, previous smaller element is 3.

For 5, previous smaller element is 4.

Ans)

The screenshot displays a coding platform interface. On the left, the 'Output Window' shows 'Compilation Results' for a problem named 'Y.O.G.J. (AI Bot)'. It indicates 'Problem Solved Successfully' with a green checkmark. Below this, it shows 'Test Cases Passed: 1115 / 1115', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 4 / 4', and 'Time Taken: 1.16'. There are buttons for 'Solve Next' and 'Stay Ahead With: Build 21 Projects in 21 Days'. On the right, the 'Java (21)' editor shows a solution for finding the previous smaller element. The code uses a stack and a deque to process an array, returning the previous smaller element for each element in the array. The code is as follows:

```
1- class Solution {
2-     public static ArrayList<Integer> prevSmaller(int[] arr) {
3-         ArrayList<Integer> result = new ArrayList<>();
4-         Deque<Integer> stack = new ArrayDeque<>(); // stores elements (not indices)
5-
6-         for (int i = 0; i < arr.length; i++) {
7-
8-             // Pop elements >= current (they can never be a "smaller" answer)
9-             while (!stack.isEmpty() && stack.peek() >= arr[i]) {
10-                 stack.pop();
11-             }
12-
13-             // Top of stack is the nearest previous smaller element
14-             result.add(stack.isEmpty() ? -1 : stack.peek());
15-
16-             // Push current element for future comparisons
17-             stack.push(arr[i]);
18-         }
19-
20-         return result;
21-     }
22- }
```

6) Given two integers n and k , the task is to find all valid combinations of k numbers that adds up to n based on the following conditions:

- Only numbers from the range $[1, 9]$ used.
- Each number can only be used at most once.

Note: You can return the combinations in any order, the driver code will print them in sorted order.

Examples:

Input: $n = 9, k = 3$

Output: $[[1, 2, 6], [1, 3, 5], [2, 3, 4]]$

Explanation: There are three valid combinations of 3 numbers that sum to 9: $[1, 2, 6]$, $[1, 3, 5]$ and $[2, 3, 4]$.

Input: $n = 3, k = 3$

Output: $[]$

Explanation: It is not possible to pick 3 distinct numbers from 1 to 9 that sum to 3, so no valid combinations exist.

Ans)

The screenshot shows a coding platform interface. On the left, a sidebar indicates the problem is solved successfully with 1120/1120 test cases passed, 1/1 attempts, 100% accuracy, 4/4 points scored, and a time taken of 0.13. The right pane displays the Java code for a backtracking solution. The code defines a `combinationSum` method that returns a list of lists of integers, and a `backtrack` method that recursively explores combinations of numbers from 1 to 9 that sum to the target `k`.

7) Given an array `arr[]` of size `n`, your task is to divide the array in two subsets such that the absolute difference between the sum of elements in the two subsets is equal to zero (i.e., both subsets have the same sum).

- If `n` is even, both subsets must contain exactly $n/2$ elements.
- If `n` is odd, one subset must contain $(n-1)/2$ elements and the other subset must contain $(n+1)/2$ elements.

Note : If multiple answers exist, you may return any of them. The driver code will check and print true if your partition is valid, otherwise false.

It is guaranteed that there will always be atleast one valid partition.

Examples:

Input: `arr[] = [1, 2, 3, 4]`

Output: `[[1, 4], [2, 3]]`

Explanation: The absolute difference between the sum of both subsets is 0.

Input: `arr[] = [5, 10, 15]`

Output: `[[5, 10], [15]]`

Explanation: The absolute difference between the sum of both subsets is 0

Ans)

The screenshot displays a coding platform interface. On the left, the 'Output Window' shows 'Compilation Results' for a problem solved successfully. It indicates that 1120 out of 1120 test cases passed, with 1 out of 3 attempts correct, an accuracy of 33%, 4 out of 4 points scored, and a time taken of 0.23. Below this, it suggests solving next problems like 'Rat in a Maze' and 'Possible Words From Phone Digits'. On the right, the Java code is shown, implementing a backtracking algorithm to find subsets of size k that maximize the minimum absolute difference between any two elements. The code uses a frequency map and a backtracking function to explore all possible subsets.

8) Given an array `arr[]` of integers and an integer `k`, select `k` elements from the array such that the minimum absolute difference between any two of the selected elements is maximized. Return this maximum possible minimum difference.

Examples:

Input: `arr[] = [2, 6, 2, 5]`, `k = 3`

Output: 1

Explanation: 3 elements out of 4 elements are to be selected with a minimum difference as large as possible. Selecting 2, 2, 5 will result in minimum difference as 0. Selecting 2, 5, 6 will result in minimum difference as $6 - 5 = 1$.

Input: `arr[] = [1, 4, 9, 0, 2, 13, 3]`, `k = 4`

Output: 4

Explanation: Selecting 0, 4, 9, 13 will result in minimum difference of 4, which is the largest minimum difference possible

Ans)

The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. It shows 'Test Cases Passed: 1115 / 1115', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 4 / 4', and 'Your Total Score: 87'. Below this, it suggests 'Solve Next' problems like 'Merge Sort', 'Quick Sort', and 'C++ Generic sort'. At the bottom, it promotes 'Build 21 Projects in 21 Days'. On the right, the code editor shows a Java solution for a problem. The code is as follows:

```
3 class Solution {
4     public int maxDiff(int[] arr, int k) {
5         Arrays.sort(arr);
6
7         int low = 0;
8         int high = arr[arr.length - 1] - arr[0];
9         int ans = 0;
10
11        while (low <= high) {
12            int mid = low + (high - low) / 2;
13
14            if (canSelect(arr, k, mid)) {
15                ans = mid; // valid + try bigger
16                low = mid + 1;
17            } else {
18                high = mid - 1; // not valid + try smaller
19            }
20        }
21        return ans;
22    }
23
24    private boolean canSelect(int[] arr, int k, int minDiff) {
25        int count = 1; // pick first element
26        int last = arr[0];
27
28        for (int i = 1; i < arr.length; i++) {
29            if (arr[i] - last >= minDiff) {
30                count++;
31                last = arr[i];
32            }
33            if (count >= k) return true;
34        }
35        return false;
36    }
37 }
38
39
40
41
42
```

9) Given a sorted array `arr[]` containing distinct non negative integers that has been rotated at some unknown pivot, and a value `x`. Your task is to count the number of elements in the array that are less than or equal to `x`.

Examples:

Input: `arr[] = [4, 5, 8, 1, 3]`, `x = 6`

Output: 4

Explanation: 1, 3, 4 and 5 are less than 6, so the count of all elements less than 6 is 4.

Input: `arr[] = [6, 10, 12, 15, 2, 4, 5]`, `x = 14`

Output: 6

Explanation: All elements except 15 are less than 14, so the count of all elements less than 14 is 6.

Ans)

The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. Below this, it shows 'Test Cases Passed: 1113 / 1113', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 4 / 4', and 'Time Taken: 0.29'. The right panel shows the Java code for finding the sum of elements in an array using a binary search approach. The code includes a function to find the index of the smallest element and a function to count elements in a sorted range.

```
16 }
17
18 // Find index of smallest element
19 private int findPivot(int[] arr) {
20     int low = 0, high = arr.length - 1;
21
22     while (low < high) {
23         int mid = low + (high - low) / 2;
24
25         if (arr[mid] > arr[high])
26             low = mid + 1;
27         else
28             high = mid;
29     }
30     return low;
31 }
32
33 // Count elements <= x in sorted range
34 private int countInRange(int[] arr, int l, int r, int x) {
35     if (l > r) return 0;
36
37     int low = l, high = r;
38     int ans = 1 - 1;
39
40     while (low <= high) {
41         int mid = low + (high - low) / 2;
42
43         if (arr[mid] <= x) {
44             ans = mid;
45             low = mid + 1;
46         } else {
47             high = mid - 1;
48         }
49     }
50
51     return ans - 1 + 1;
52 }
53
54 }
55
```

10) You will be given a vector of integers of size n containing the elements. Your task is to find the sum of all the integers present in the array. As the sum can be large you have to return a value in long long data type.

Example 1:

Input:

$n=7$

input= {6,2,5,4,5,1,6}

Output : 29

Ans)



Search...

Sign Out

Courses

Tutorials

Practice

Jobs

zA

A

Problem

Editorial

Submissions

Comments

Output Window

Compilation Results

Custom Input

Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1 / 1

Attempts : Correct / Total
1 / 1

Accuracy : 100%

Points Scored
1 / 1

Your Total Score: 92

Time Taken
0.06

Solve Next

Replace all 0's with 5

Largest product

Third Largest

Stay Ahead With:



Build 21 Projects in 21 Days

Build real-world ML, Deep Learning & Gen AI projects

Java (21)

Start Timer

1 class Solution {
2 public long get_Sum(int n, int[] input) {
3 long sum = 0;
4
5 for (int i = 0; i < n; i++) {
6 sum += input[i];
7 }
8 return sum;
9
10 }
11 }

Custom Input

Compile & Run

Submit